

Retail Sales Analytics Using Excel, SQL, Power BI and Python

Problem Statement:

In a competitive retail market, businesses need to track sales performance, understand customer behavior, and identify growth opportunities. This project aims to build an end-to-end analytics solution that enables insights generation from raw transactional data using a multi-tool approach.

Dataset Collection:

Datasets:

- Brands
- Categories
- Customers
- Orders
- Order_items
- Products
- Staffs
- Stocks
- Stores

Dataset Size:

Total Tables:6

Total Attributes:54

Excel Data Cleaning and Preparation Report

1. Introduction

Microsoft Excel was used as the initial stage of the data analytics project. The raw datasets contained information related to customers, orders, order items, products, stores, staff, brands and categories. Before importing the data into SQL, the datasets were cleaned, standardized and validated in Excel.

The main purpose of this stage was to improve data quality, maintain consistency between different sheets and prepare the data for further SQL, Python and Power BI analysis.

2. Standardize Column Headers

The first step was to standardize the column names across all worksheets. Consistent column names make it easier to identify fields and perform operations when transferring the data to SQL and Python.

For example, column names were maintained in a consistent format such as:

Original/Example	Standardized
Customer ID	customer_id
Order ID	order_id
Product ID	product_id
Order Date	order_date
List Price	list_price
Quantity	quantity
Order Status	order_status

The snake_case naming convention was preferred because it is commonly used in SQL and Python.

3. Remove Duplicates

Duplicate records were checked in important worksheets such as customers, orders and order_items.

In Excel, duplicate records can be identified using:

Data → Remove Duplicates

Before removing duplicates, the data was checked carefully because some repeated values are legitimate. For example, an order_id can appear multiple times in the order_items table because one order can contain multiple products.

Therefore, duplicate removal was performed based on the appropriate combination of columns rather than simply deleting every repeated order_id.

4. Handle Missing Values

Missing values were identified using Excel filters and blank-cell checking.

The following types of fields were reviewed:

- Customer information
- Phone numbers
- ZIP/postal codes
- Product information
- Dates
- Other important categorical fields

Excel filters were used to identify blank values.

For fields where the actual information was unavailable, a suitable placeholder such as "NA" could be used where appropriate.

For example:

customer_id	phone	zip_code
101	9876543210	641006
102	NA	641007

Care was taken not to replace missing numerical values with inappropriate values such as 0, because zero and missing are not necessarily the same.

5. Data Type Conversion

Correct data types are important for calculations and analysis.

Date fields

Fields such as:

- order_date
- shipped_date
- required_date

were converted into proper Excel Date format.

This allows operations such as:

- Sorting by date
- Calculating date differences

- Extracting year/month
- Creating time-based analysis

Numeric fields

Numerical fields such as:

- quantity
- list_price
- discount
- total_price

were checked and converted to appropriate Number formats.

For example:

quantity → Whole Number

list_price → Number/Currency

This ensures that mathematical calculations can be performed correctly.

6. Data Validation

Data Validation was used to maintain consistency when entering categorical values.

For fields such as order_status, a predefined dropdown list can be created.

In your dataset, the order status is represented by 1, 2, 3 and 4. If these codes correspond to statuses such as Pending, Processing, Shipped and Delivered, the dropdown can use the appropriate status values.

The purpose of the dropdown is to prevent inconsistent entries such as:

- Pending
- pending
- PENDING
- Pendding

Instead, users select one valid predefined value.

7. Create New Derived Columns

A new column called **total_price** was created in the order_items worksheet.

The calculation is:

Total Price = (List Price × Quantity) – Discount

This derived field is important because it provides the transaction-level revenue value used later for sales and revenue analysis.

8. Merge Lookup Data

Product information was merged into the order_items worksheet using product_id.

The purpose was to retrieve information such as the **product name** from the Products sheet.

For example:

product_id	product_name
101	Mountain Bike
102	Road Bike

An XLOOKUP formula can be used:

=XLOOKUP(A2,Products!A:A,Products!B:B,"NA")

Here:

- A2 = product_id in order_items
- Products!A:A = product_id in the Products sheet
- Products!B:B = product_name

If XLOOKUP is unavailable, VLOOKUP can also be used.

9. Create Basic Pivot Table

A Pivot Table was created to summarize sales performance by product category.

The Pivot Table can be configured as:

Rows:

- Category Name

Values:

- Sum of Total Price

Example:

This provides a quick comparison of revenue generated by different product categories.

The Pivot Table also acts as an initial validation of the sales calculations before moving to SQL and Power BI.

10. Sort and Filter for Outliers

The list_price field was examined to identify unusually high or low product prices.

Excel sorting and filtering were used to identify:

- Very high-priced products
- Very low-priced products

- Unusual price values

For example, sorting list_price from Largest to Smallest makes it easy to identify products with exceptionally high prices.

These values were not automatically deleted because a high price may represent a genuine premium product rather than an error.

The outlier analysis was later explored further using Python and a boxplot with the IQR method.

11. Prepare Final CSV Files

After completing the cleaning and validation process, each worksheet was prepared for SQL import.

The cleaned datasets were saved as separate CSV files, for example:

- customers.csv
- orders.csv
- order_items.csv
- products.csv
- brands.csv
- categories.csv
- stores.csv
- staff.csv

Each file contains a cleaned and standardized dataset that can be imported into MySQL.

Total price	Product_name	xlookup	Category_id	Category_name		
599.79	Electra Townie Original 7D EQ - Women's - 2016	Electra Townie Original 7D EQ - Women's - 2016	3	Cruisers Bicycles		
3599.91	Trek Remedy 29 Carbon Frameset - 2016	Trek Remedy 29 Carbon Frameset - 2016	6	Mountain Bikes		
3097.95	Surly Straggler - 2016	Surly Straggler - 2016	4	Cyclocross Bicycles		
1199.93	Electra Townie Original 7D EQ - 2016	Electra Townie Original 7D EQ - 2016	3	Cruisers Bicycles		
2899.79	Trek Fuel EX 8 29 - 2016	Trek Fuel EX 8 29 - 2016	6	Mountain Bikes		
599.92	Electra Townie Original 7D EQ - Women's - 2016	Electra Townie Original 7D EQ - Women's - 2016	3	Cruisers Bicycles	Row Labels	Sum of Total price
1199.93	Electra Townie Original 7D EQ - 2016	Electra Townie Original 7D EQ - 2016	3	Cruisers Bicycles	Children Bicycles	327803.83
999.94	Surly Wednesday Frameset - 2016	Surly Wednesday Frameset - 2016	6	Mountain Bikes	Comfort Bicycles	438452.02
599.94	Electra Townie Original 7D EQ - Women's - 2016	Electra Townie Original 7D EQ - Women's - 2016	3	Cruisers Bicycles	Cruisers Bicycles	1109007.23
1499.88	Ritchey Timberwolf Frameset - 2016	Ritchey Timberwolf Frameset - 2016	6	Mountain Bikes	Cyclocross Bicycles	799846.1
3097.95	Surly Straggler - 2016	Surly Straggler - 2016	4	Cyclocross Bicycles	Electric Bikes	1020214.56
428.93	Pure Cycles Vine 8-Speed - 2016	Pure Cycles Vine 8-Speed - 2016	3	Cruisers Bicycles	Mountain Bikes	3030651.45
599.92	Electra Townie Original 7D EQ - 2016	Electra Townie Original 7D EQ - 2016	2	Comfort Bicycles	Road Bikes	1852516.12
448.93	Pure Cycles Western 3-Speed - Women's - 2015/21	Pure Cycles Western 3-Speed - Women's - 2015/2016	3	Cruisers Bicycles	Grand Total	8578491.31
1099.93	Electra Townie Original 21D - 2016	Electra Townie Original 21D - 2016	3	Cruisers Bicycles		
599.89	Electra Townie Original 7D EQ - Women's - 2016	Electra Townie Original 7D EQ - Women's - 2016	3	Cruisers Bicycles		
1999.91	Surly Wednesday Frameset - 2016	Surly Wednesday Frameset - 2016	6	Mountain Bikes		

SQL – Database Management and Querying

MySQL was used as the second stage of the data analytics workflow after the data was cleaned and prepared in Excel. The main purpose of this stage was to organize the datasets into a

relational database, establish relationships between tables, and perform business-oriented analysis.

1. Database Creation and ER Diagram Implementation

The database was designed based on the Entity Relationship Diagram (ERD). The ERD represents the entities involved in the retail sales system and the relationships between them.

The major entities used in the database include:

- **Customers** – stores customer details.
- **Orders** – stores information about customer orders.
- **Order Items** – stores individual products included in each order.
- **Products** – contains product details such as product name, price and category.
- **Brands** – stores information about product brands.
- **Categories** – classifies products into different categories.
- **Stores** – contains store-related information.
- **Staff** – stores employee/staff information.

The relationships between these tables are established using primary keys and foreign keys.

For example:

- **Customers → Orders**
 - A customer can place multiple orders, so `customer_id` connects the Customers table with the Orders table.
- **Orders → Order Items**
 - An order can contain multiple products, so `order_id` connects Orders with Order Items.
- **Products → Order Items**
 - A product can appear in multiple order items, so `product_id` connects Products with Order Items.
- **Products → Brands**
 - Each product belongs to a brand, so `brand_id` connects Products with Brands.
- **Products → Categories**
 - Each product belongs to a category, so `category_id` connects Products with Categories.
- **Stores → Orders**
 - Orders are associated with the store where they were handled.
- **Staff → Orders**
 - Orders are also associated with the staff member responsible for handling them.

2. Data Import and Database Preparation

After completing the Excel data-cleaning process, the cleaned CSV files were imported into MySQL using MySQL Workbench.

The tables were created according to the ERD, with appropriate:

- Primary keys
- Foreign keys
- Data types
- Constraints

The cleaned CSV files were then loaded into their respective tables.

Data validation was performed after importing the files to ensure that:

- Records were loaded correctly.
- Column data types were appropriate.
- Primary key values were valid.
- Foreign-key relationships were maintained.
- The number of records matched the cleaned source data.

This created a structured relational database that could be used for further analysis.

3. SQL-Based Business Analysis

Once the database was populated, SQL was used to answer important business questions.

Order and Product Analysis

The related Orders, Order Items and Products tables were combined to obtain detailed order information. This made it possible to analyze individual products purchased within each order and calculate sales-related measures.

The analysis included identifying the top-selling products based on quantity, allowing products with the highest demand to be recognized.

Store Sales Analysis

Sales were summarized based on store_id to determine the total revenue generated by each store.

This helps compare store performance and identify locations contributing the most to overall revenue.

Customer Purchase Summary

Customer-level analysis was performed to calculate:

- Total orders placed

- Total items purchased
- Total revenue generated

This provides an overall view of customer purchasing behavior and helps identify high-value customers.

Customer Spending Classification

Customers were also classified into spending categories such as:

- **Low spending**
- **Medium spending**
- **High spending**

This classification provides a simple customer-value view and can support targeted marketing strategies.

4. Staff, Inventory and Customer Segmentation

Staff Performance Analysis

Staff performance was analyzed based on the orders handled by each staff member. Revenue associated with their handled orders was summarized to identify staff members contributing strongly to sales.

This helps management evaluate employee performance and understand revenue contribution by staff.

Stock Alert Analysis

Inventory levels were analyzed to identify products with stock quantities below 10 units in any store.

These products can be treated as potential stock alerts and may require:

- Reordering
- Inventory monitoring
- Store-level stock transfers

This analysis helps prevent stock shortages.

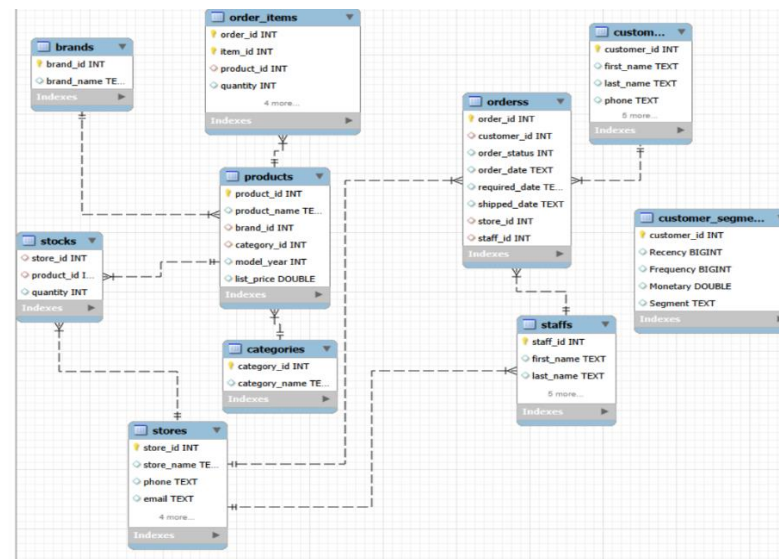
Customer Segmentation Table

A separate table named customer_segments was created in MySQL to store the customer segmentation results generated later using Python.

The table was designed to contain customer-level information such as:

- customer_id
- Recency
- Frequency
- Monetary value
- Customer segment

The segmentation results generated through Python were subsequently inserted into this table and used in Power BI for customer-segment analysis.



```
Navigation | queries | text functions | practice Sql que | joins | windows function | sql mini project | capstone.sql | x
Limit to 1000 rows | Filter Rows | Export | Wrap Cell Contents |

72 on oi.order_id=oi.order_id
73 inner join staffs st
74 on o.staff_id=st.staff_id
75 group by st.staff_id;
76
77 -- 9. Stock Alert Query
78 -- Write a query to list products where stock quantity < 10 in any store.
79 • select p.product_id, p.product_name,sto.quantity from capstone.products p
80 inner join capstone.stocks sto
81 on p.product_id=sto.product_id
82 where sto.quantity<10;
83 -- 10. Create Final Segmentation Table
84 -- Create a table customer_segments that will be populated from Python ML results later.
85
86
87
```

product_id	product_name	quantity
2	Ritchey Trailblazer Frameset - 2016	5
3	Surly Wednesday Frameset - 2016	6
6	Surly Ice Cream Truck Frameset - 2016	0
7	Trek Slash 8 27.5 - 2016	8
8	Trek Remedy 29 Carbon Frameset - 2016	0

Result 1 x

Output

Action Output

#	Time	Action	Message
1	21:20:57	use capstone	0 row(s) affected
2	21:21:11	select p.product_id, p.product_name,sto.quantity from capstone.products p inner join capstone.stocks sto on p.p...	323 row(s) returned

Python Tasks

1. Loading Data from SQL

The cleaned and structured data stored in MySQL was imported into Python for further analysis. Pandas was used to retrieve the required tables from the SQL database and store them as DataFrames.

The main tables used for the analysis were:

- **Orders** – used to obtain order dates and customer-order relationships.
- **Order Items** – used to calculate quantities, prices and purchase values.
- **Customers** – used to obtain customer-level information.

The `pandas.read_sql()` function was used to retrieve the SQL data into Python. Loading the data directly from SQL ensured that the Python analysis was performed on the structured and validated database rather than on the original raw files.

2. Exploratory Data Analysis (EDA)

Exploratory Data Analysis was performed to understand the structure, quality and distribution of the dataset before calculating customer-level features.

Dataset Inspection

Functions such as:

- `df.info()`
- `df.describe()`
- `df.value_counts()`

were used to examine the dataset.

`df.info()` was used to understand:

- Number of records
- Column names
- Data types
- Missing values

`df.describe()` was used to obtain statistical summaries of numerical fields such as:

- Quantity
- List price
- Total price
- Other numerical measures

`df.value_counts()` was used to understand the frequency of categorical values such as order status, categories and other relevant fields.

Data Visualization

Matplotlib and Seaborn were used to visually explore the data.

The analysis included charts such as:

- Bar charts for categorical comparisons
- Distribution plots for numerical variables
- Boxplots for identifying potential outliers

A boxplot of list_price was used to examine the price distribution. The visualization showed that most products were concentrated within a lower price range, while some products had considerably higher prices and appeared as potential outliers.

These observations helped in understanding the characteristics of the dataset before proceeding with customer segmentation.

3. Calculation of RFM Features for Customers

RFM analysis was used to evaluate customer purchasing behavior based on three important characteristics:

Recency

Recency measures how recently a customer made a purchase.

It was calculated as the number of days between the customer's most recent order date and the selected reference date.

A lower Recency value indicates that the customer purchased more recently.

Frequency

Frequency measures how often a customer places orders.

It was calculated using the number of orders associated with each customer. Unique order_id values were considered so that multiple products within the same order were not incorrectly counted as separate orders.

A higher Frequency value indicates that the customer purchases more frequently.

Monetary

Monetary measures the total value generated by a customer.

It was calculated by summing the purchase value associated with the customer's orders/order items.

A higher Monetary value indicates that the customer contributes more revenue to the business.

RFM Summary

RFM Feature	Measures	Business Interpretation
Recency	Days since last purchase	How recently the customer purchased
Frequency	Number of orders	How frequently the customer purchases
Monetary	Total purchase value	How valuable the customer is in terms of spending

The three measures were combined to create a customer-level RFM dataset.

4. Customer Segmentation

After calculating the RFM values, customers were assigned segment labels based on their purchasing behavior.

The segmentation helps divide customers into meaningful groups such as:

- **Champions**
- **Loyal Customers**
- **Potential Loyalists**
- **At Risk**
- **Others**

The purpose of segmentation is to understand different customer groups rather than treating every customer in the same way.

For example:

- **Champions** can be targeted with loyalty rewards and premium offers.
- **Loyal Customers** can be encouraged to make repeat purchases.
- **Potential Loyalists** can be targeted with offers designed to increase their purchase frequency.
- **At Risk** customers can be targeted with retention campaigns.
- **Others** can be analyzed further to identify opportunities for conversion.

The resulting RFM dataset contained customer-level values along with the corresponding segment label.

5. Exporting Segmentation Results to SQL

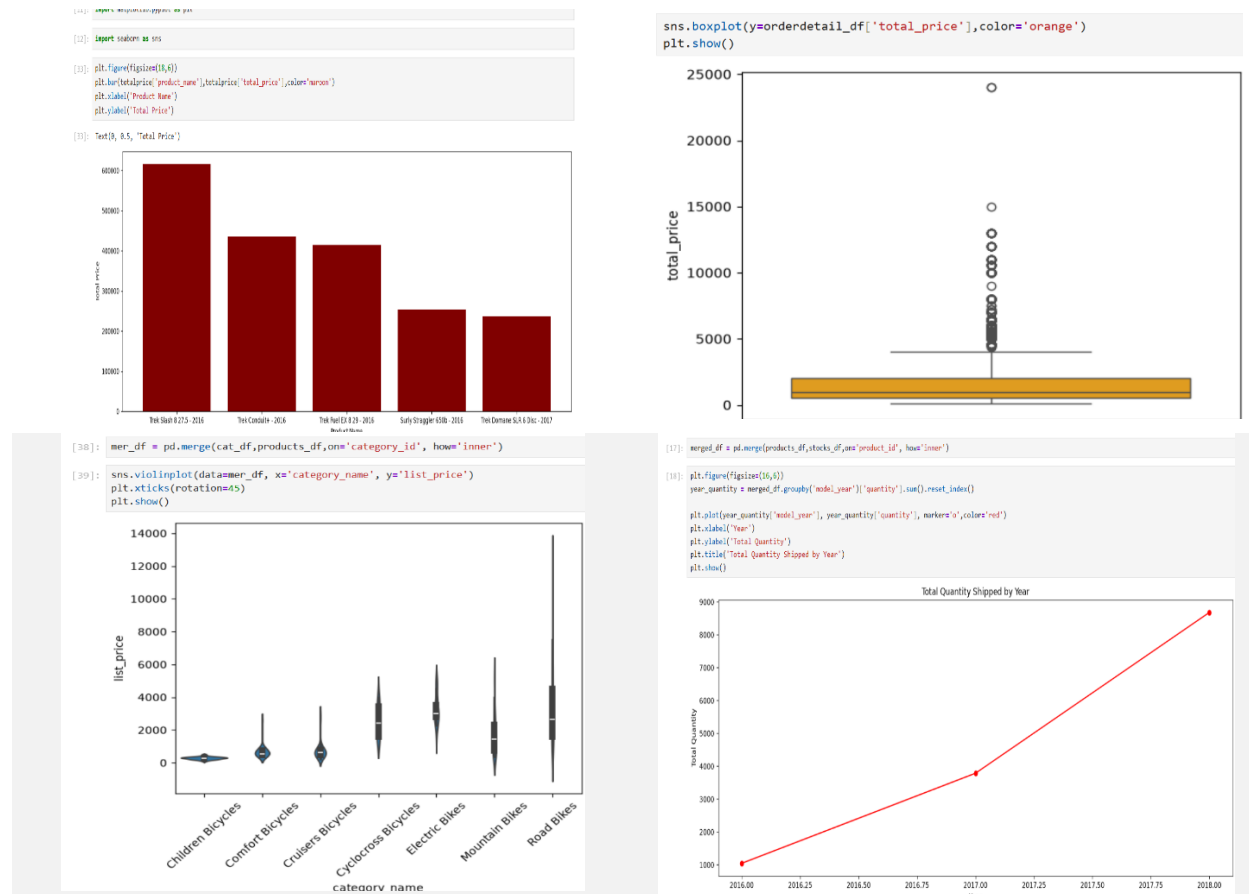
After completing the RFM calculation and assigning segment labels in Python, the resulting dataset was exported back to MySQL.

A table named `customer_segments` was created to store the segmentation results.

The table contains information such as:

- customer_id
- recency
- frequency
- monetary
- segment

The customer segmentation results were then available in SQL for further querying and were also used in Power BI to analyze revenue and customer performance by segment.



Power BI – Visualization & Dashboarding

Power BI was used as the final visualization and reporting stage of the project. The cleaned and analyzed data from MySQL and the customer segmentation results generated using Python were connected to Power BI to create interactive dashboards and business reports.

1. Connect Power BI to SQL and Create Data Model

The key tables from the MySQL database were imported into Power BI, including:

- Customers
- Orders
- Order Items
- Products
- Stores
- Staff
- Brands
- Categories
- Customer Segments

The relationships between the tables were created using Model View based on the ER diagram.

For example, customer_id was used to establish the relationship between the Customers and Orders tables. Similarly, order_id connected Orders with Order Items, while product_id connected Order Items with Products.

This relational model allowed information from multiple tables to be analyzed together without unnecessarily duplicating data.

2. Sales and Product Performance Analysis

A Sales Overview Report was created to provide an overall view of business performance.

The dashboard included:

- **Total Revenue** – shows the overall revenue generated.
- **Total Orders** – shows the number of orders placed.
- **Average Order Value** – indicates the average revenue generated per order.
- **Sales Trend** – displays how sales change over time using a line or area chart.
- **Monthly Sales Trend** – helps identify months with higher or lower sales.

Product performance was also analyzed using bar charts to identify the top-selling products based on revenue or quantity sold. These visuals help management understand overall sales performance and identify products contributing significantly to revenue.

3. Customer, Store and Staff Analysis

Customer Purchase Analysis

Customer purchasing patterns were visualized using customer-related information such as city and state.

Stacked bar charts were used to compare purchasing activity across different locations. This helps identify areas with higher customer activity and sales contribution.

Sales by Store

A map visual was used to display the geographical distribution of sales based on store location information such as state or ZIP code.

This allows the performance of different store locations to be compared geographically.

Staff Performance

Staff performance was analyzed by calculating the total revenue generated from the orders handled by each staff member.

A chart/table was used to compare staff members based on their sales contribution.

This provides management with a clear view of individual staff performance.

4. Inventory Monitoring and Interactive Filtering

A Low Stock Alert Dashboard was created to identify products where the available stock quantity falls below the defined threshold of 10 units.

Conditional formatting was used to highlight products requiring attention, while KPI cards were used to provide a quick summary of low-stock items.

Interactive slicers were also added to allow users to filter the dashboard based on:

- **Order Status**
- **Product Category**
- **Store**

These filters allow users to investigate specific parts of the business without changing the underlying data.

5. Customer Segmentation Analysis

The customer_segments table created through Python was imported from MySQL into Power BI.

This table contained the RFM-based customer information, including:

- Customer ID
- Recency

- Frequency
- Monetary value
- Customer Segment

The customer segment distribution was visualized using a TreeMap, where each segment represents a category and its size represents the number of customers in that segment.

For example, segments could include:

- Loyal Customers
- Potential Loyalists
- At-Risk Customers
- Champions
- Others

A bar chart was also created to show Total Revenue by Customer Segment. This helps identify which customer groups contribute the most revenue.

A customer-segment slicer was added so that the other dashboard visuals could be analyzed based on the selected customer segment.

6. Consolidated Dashboard

A final consolidated dashboard page was created to bring the major business metrics and visualizations together in one interactive report.

The main KPI cards included:

KPI	Purpose
Total Revenue	Measures overall sales revenue
Active Customers	Shows the number of customers with purchases
Average Order Value	Measures average revenue per order
Total Orders	Shows the total number of orders

The dashboard also incorporated sales trends, product performance, customer analysis, store performance, staff performance, inventory alerts and customer segmentation.

Interactive buttons, bookmarks and slicers were used to improve dashboard navigation and allow users to switch between different views of the report.

Overall Power BI Outcome

The Power BI dashboard transformed the cleaned and analyzed data into an interactive business intelligence report. It enables users to monitor sales, understand customer purchasing behavior, compare store and staff performance, identify low-stock products and analyze customer segments from a single reporting environment.

